דו״ח סיכום האקתון RISCV

איתמר מורג – 316526557, ניתאי גילדור – 318657780

1. מבוא

הבעיה שאיתה התמודדנו בהאקתון עוסקת בהתאמת רצפי DNA (DNA Sequence Alignment).

ה DNA מיוצג על ידי ארבעה נוקלאוטידים (A, C, G, T) ומציאת דמיון בין רצפי DNA היא משימה קריטית ליישומים שונים, כגון ריצוף גנטי, חקר מחלות, איתור מוצא (ancestry) והרכבת גנום.

השוואת מחרוזות פשוטה ומדויקת אינה מספיקה במקרה זה, משום שייתכנו מוטציות (חוסר התאמה בין בסיסים) או בסיסים חסרים המייצגים אירועי הכנסה או מחיקה (Insertion/Deletion) כדי להתמודד עם אתגר זה, נעשה שימוש באלגוריתם Smith-Waterman המיועד למציאת ההתאמה המקומית (Local Alignment) הטובה ביותר בין שני רצפים.

האלגוריתם עושה שימוש במודל ניקוד מסוג Affine Gap Penalty, שהוא מציאותי יותר מבחינה ביולוגית כיוון שהוא מחמיר יותר עם פתיחת רווח חדש מאשר עם הרחבת רווח קיים.

המשימה המרכזית באתגר היא קבלת רצף שאילתה (Query) אחד ומספר רצפי ייחוס (References) וחישוב ציון ההתאמה המקומית הגבוה ביותר עבור כל רצף ייחוס מול השאילתה.

מטרות הפרויקט נגזרות מדרישות האתגר ומארכיטקטורת הפתרון הייחודית שבחרנו לממש:

פיתוח חומרה ייעודית: תכנון ופיתוח של מאיץ חישובי על גבי FPGA העובד יחד עם מעבד מבוסס RISC-V.

מקסום ביצועים (Latency): מזעור זמן הריצה של משימת החישוב הכוללת. השגנו זאת על ידי יצירת 8 ליבות חומרה עצמאיות הפועלות במקביל, מה שמאפשר הערכה של 8 רצפי ייחוס מול השאילתה בו-זמנית.

ייעול משאבים (Area): שימוש מינימלי ויעיל במשאבי החומרה של ה-FPGA (למשל צמצום רגיסטרים) במסגרת זו, שאפנו לממש את האלגוריתם כך שישתמש רק בבלוקים הנדרשים בכל שלב חישוב במקום לשמור את המטריצה כולה, ובכך לחסוך במשאבים.

ייעול תעבורת הנתונים (I/O): מזעור צווארי בקבוק בתקשורת הנתונים (BUS) על ידי אריזה (Packing) של רצפי ה-DNA לתוך מילים של 32-bit, דחיסת התוצאות הסופיות ודחיסת אורך ברצפים למספר רגיסטרים מינימלי.

2. תיאור הפתרון

תהליך הפיתוח שלנו התבצע בצורה איטרטיבית לאורך ההאקתון:

א. אופטימיזציות תוכנה (Spatial Memory Efficiency)

התחלנו בעבודה עם קוד ה C-המקורי. זיהינו ששמירת שלוש מטריצות שלמות (32X32) גורמת לבזבוז משאבי זיכרון ומובילה לקריאות/כתיבות מרובות ל-Cache.

ייעלנו את האלגוריתם כך שינצל את העובדה שחישוב כל תא תלוי רק בשורה הנוכחית ובשורה הקודמת. במקום לשמור את המטריצות במלואן, שמרנו רק שורה אחת קודמת עבור I, ושמרנו משתנה בודד המייצג את הערך הקודם של D (זה מה שנדרש לחישוב על תא באלגוריתם Affine Smith-Waterman). שיפור זה הוריד את סיבוכיות המקום וסיפק לנו שיפור ביצועים (Speedup) של פי 1.5 עוד לפני שנגענו בחומרה.

ב. מעבר להאצת חומרה(Single Core)

כדי להשיג שיפור משמעותי יותר, החלטנו לממש את האלגוריתם המיועל במלואו בתוך מאיץ חומרה. בנינו את המודול smith_waterman_core.svאשר מתפקד כמכונת מצבים (FSM) ומחשב את הציונים בצורה סידרתית אך מיועלת מאוד:

- בכל במחזור שעון המערכת מחשבת תא אחד. מבנה ה FSM שלנו מחזיק בלוק קומבינטורי שמחשב לפי הנוסחאות הנתונות את הערך של התא
- בהתבסס על השיפור התוכנתי, הליבה שומרת רק את השורה הקודמת (M_prev ו I_prev במערכים חד מימדיים) ורגיסטרים בודדים לערכים השמאליים והאלכסוניים (m_left, d_left, m_diag).

שילוב הליבה הבודדת במאיץ הבסיסי הוריד את זמן הריצה לכ-15,000 מחזורי שעון, והוכיח שהארכיטקטורה יציבה ותקינה (לרצפים באורך 16 תווים).

ג. מקביליות - ארכיטקטורת 8 ליבות(8-Way ParallelProcessing)

לאחר שווידאנו שהליבה הבודדת פועלת כראוי, עברנו למימוש מקביליות. שכפלנו את ליבת ה-Smith Waterman ל-8 ליבות חומרה נפרדות בתוך המאיץ. כל הליבות מקבלות את אותו רצף שאילתה (Query) ואת פקודת ההפעלה בו-זמנית, אך כל ליבה מחשבת את ההתאמה מול רצף ייחוס (Reference) שונה. התאמנו את קוד ה C כך שיזין את כל 8 הרצפים בבת אחת לפי תחילת פעולת המאיץ.

ד. אופטימיזציית I/O ודחיסת נתונים (Data Packing)

גילינו שהכתיבה והקריאה גוזלות מחזורי שעון יקרים. כדי לפתור זאת, צמצמנו למינימום את מספר הרגיסטרים המעורבים בתקשורת:

דחיסת קלט :כל נוקלאוטיד ניתן לייצוג על ידי 2 ביטים בלבד. לכן, רצף DNA של 16 תווים נדחס למילה בודדת של 32-bit. הדחיסה מתבצעת ברמת המעבד (pack_sequence) באמצעות פעולות Bitwiseמהירות ללא לולאות.

דחיסת פלט :הציון המקסימלי האפשרי לרצף של 16 תווים דורש לכל היותר 6 ביטים. במקום לבצע 8 קריאות נפרדות מהמאיץ, ארזנו את כל 8 התוצאות לתוך 2 רגיסטרים בלבד (4 תוצאות בכל רגיסטר של 32-bit).

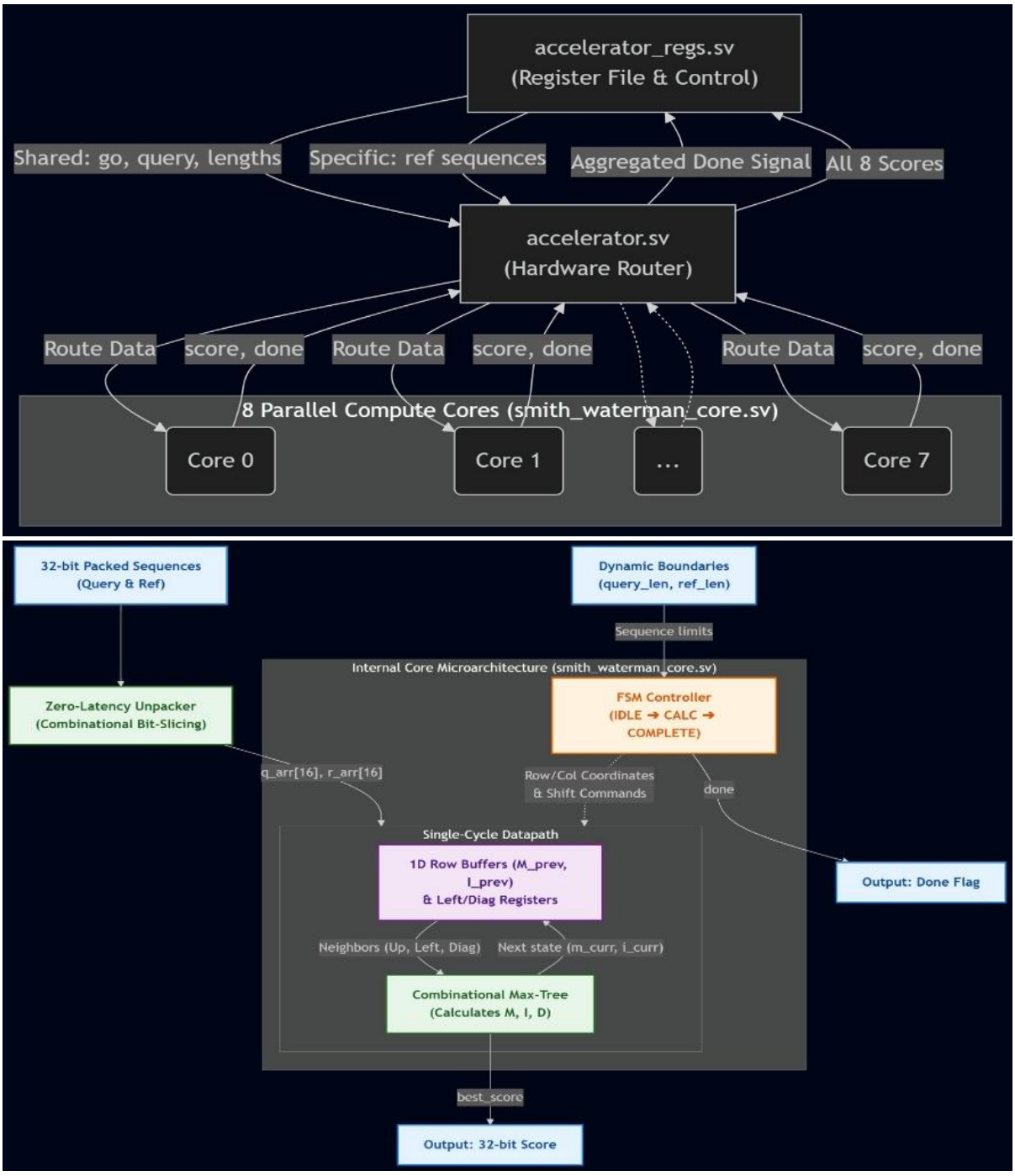
ה. תמיכה באורכים משתנים ועצירה מוקדמת (Dynamic Early Stop)

במקום להניח תמיד שהמחרוזות הן באורך 16, רצינו שהמערכת תהיה גנרית וכללית יותר.

הוספנו תמיכה בהעברת האורכים המדויקים של השאילתה ושל רצפי הייחוס לתוך המאיץ (אורכים שנדחסו לתוך 2 רגיסטרים ייעודיים). מכונת המצבים של החומרה קוראת את הגבולות הדינמיים האלו (query_len ו ref_len) ועוצרת את החישוב ברגע שהגיעה לקצה המחרוזת בפועל.

למרות שתמיכה זו מוסיפה תקורה זניחה של חישובי חומרה למחרוזות באורך מלא, היא הופכת את המערכת לעמידה, מדויקת הרבה יותר לפי דרישות האתגר, וחוסכת מחזורי שעון משמעותיים כאשר המחרוזות קצרות. עבור המחרוזות בדיקה הנתונות עלינו מ 2431 מחזורים ל 3469 מחזורים.

דיאגרמת בלוקים (למעלה של כל המאיץ, למטה של כל ליבה):



3. סביבת הפיתוח ותוצאות הפרויקט

סביבת הפיתוח פיתוח החומרה בוצע ב Vivado באמצעות תבנית המאיץ הנתונה בשילוב מעבד ה RISC-V. סביבת התוכנה נוהלה ב Segger Embedded Studio.

תוצאות האצה: זמן ביצוע כולל במחזורי שעון: 3469
משאבי חומרה:

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	DSPs (240)
N rvfpganexys	41153	22109	878	316	12719	39985	1168	44	4
clk_gen (clk_gen_nexys)	1	2	0	0	1	1	0	0	0
dmi_wrapper (dmi_wrapper)	62	98	0	0	36	62	0	0	0
ram (axi_ram)	126	130	0	0	65	126	0	32	0
swervolf (veerwolf_core)	40697	21797	878	316	12599	39529	1168	11	4
vga_mem (axi_mem)	233	66	0	0	85	233	0	1	0

בדיקת התוצאות הפלט אומת מול סט הנתונים המקורי להבטחת זהות מלאה לתוכנית ה C הנדרשת. בנוסף בוצעו בדיקות עם מחרוזות קצרות מ 16 תווים ומספר רצפים משתנה. בדיקות אלו אישרו שמנגנון העצירה הדינמית פועל כראוי ויודע להתאים את החישוב לאורך בפועל ללא פגיעה בדיוק.

4. סיכום

המאיץ שפיתחנו מאפשר חישוב מהיר יותר פי 333 לעומת שימוש במעבד הבסיסי הנתון.

<u>חשיבה ביקורתית:</u>

<u>נקודות טובות:</u>

- ייעול האלגוריתם מבחינת לוקאליות זכרון וצמצום החומרה הנדרשת.
- גנריות שמאפשרת שכפול והרחבה למספר ליבות רב בקלות.
- שימוש בכמות רגיסטרים מינימלית להעברת המידע.

<u>נקודות לייעול:</u>

- בדיעבד יכול להיות שהיה יותר מהיר לעשות את דחיסת הביטים במאיץ למרות מחיר הכתיבה הנוספת.
- הפתרון הנפוץ השתמש במערכים סיסטולים על מנת לצמצם את כמות המחזורים הנדרשת לכל חישוב (בצורה מודעת בחרנו לנסות את הגישה שלנו עם האלגוריתם המיועל שלנו).
- בקוד C חישבנו בו את כתובות הזכרון בזמן הריצה – בדיעבד היה נכון יותר לשמור אותם כקבועים.